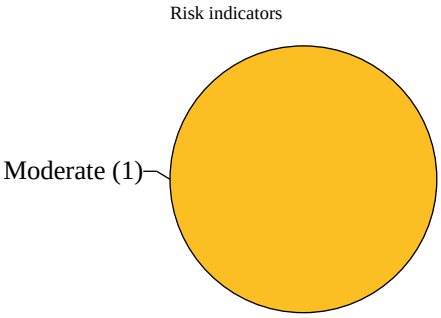
CodeGuard AI: Code Similarity Report

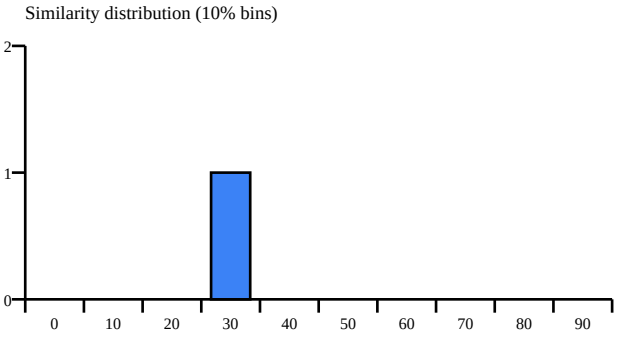
Generated 2026-10-23 10:46. 1 pair(s) compared.

Similarity scores are indicators for human review. They are not proof of plagiarism: shared assignment templates, common algorithms, boilerplate and short programs can all produce high scores, and copied code that was heavily restructured can score low.

Analysis summary

1 pair(s) from 2 file(s) were compared. 0 pair(s) are in the High indicator band, 1 Moderate and 0 Low; the average similarity is 33.2%. The highest pair is ansh__python_assignment_2_alternative.py and riya__python_programming_assignment_2.py at 33.2%.





Ranking

File A	File B	Overall	Risk indicator	Matched lines	Blocks
ansh__python_assignment_2_alternative.py	riya__python_programming_assignment_2.py	33.2%	Moderate	0	0

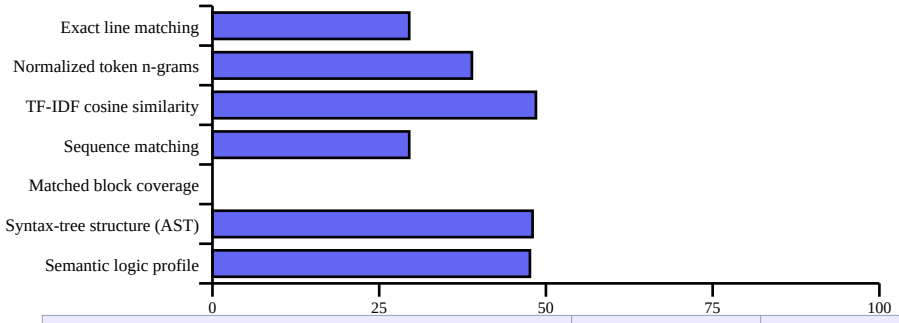
ansh__python_assignment_2_alternative.py vs riya__python_programming_assignment_2.py

33.2% Moderate similarity indicator. 0 matching line(s) in 0 block(s). Coverage: A 0.0%, B 0.0%.

Method: Python AST + normalized token analysis. Python syntax trees were compared in addition to the lexical, sequence and semantic signals.

Why this score

- Overall 33.2% (Moderate indicator) is a weighted blend of 7 signals. Largest contributions: Syntax-tree structure (AST) 48%, Semantic logic profile 48%, TF-IDF cosine similarity 48%.
- No run of 3 or more matching consecutive lines was found.
- Syntax-tree structure overlaps 48%; 0 function(s) have matching structure.
- Excluded as non-evidence (false-positive reduction): A: 3 entry point; B: 3 entry point.
- Unique code: 33 comparable line(s) in A and 28 in B have no match.



Signal	Score	Weight
Exact line matching	29.5%	10%
Normalized token n-grams	38.9%	12%
TF-IDF cosine similarity	48.5%	10%
Sequence matching	29.5%	12%
Matched block coverage	0.0%	20%
Syntax-tree structure (AST)	48.0%	18%
Semantic logic profile	47.6%	18%

Unique code (no match in the other file)

A: 33 comparable line(s) in ranges 4-48.

B: 28 comparable line(s) in ranges 4-39.

Ignored as non-evidence: A: 2 imports, braces and docstrings, 3 entry point; B: 2 imports, braces and docstrings, 3 entry point.

Methodology and limitations

Exact line matching: Dice overlap of code lines that are identical after comments and whitespace are removed. Sensitive to copy-paste; blind to renaming.

Normalized token n-grams: Dice overlap of 4-token sequences in which identifiers, numbers and strings are replaced by placeholders. Resistant to renaming and reformatting; also matches generic language idioms.

TF-IDF cosine similarity: Cosine similarity of TF-IDF vectors built from identifiers, literals and structural token trigrams. Rare shared terms weigh more than common ones when several files are analysed together.

Sequence matching: difflib SequenceMatcher ratio over the ordered, identifier-normalized code lines. Rewards code that keeps its order.

Matched block coverage: Share of significant lines covered by matched blocks found with Greedy String Tiling, which also finds blocks that were moved.

Syntax-tree structure (AST): Dice overlap of node-type trigrams of the syntax tree (names, literal values and docstrings ignored). Python uses the real ast module; Java, C, C++ and JavaScript use a lightweight statement-level parser that builds an approximate tree. Same-language pairs only.

Semantic logic profile: Local semantic fingerprint: code is rewritten into a canonical form (compound assignments expanded, loop styles unified, types and modifiers dropped, I/O and collection calls mapped to common names) and compared statement by statement. Finds the same logic written with different variable names, loop styles or even languages. This is a rule-based model that runs fully offline, not a neural network.

The overall score is a weighted mean of the available signals; weights are configurable and re-normalized when a signal does not apply. Risk levels use configurable thresholds and are not universally valid cut-offs. Comments, whitespace, import lines, brace-only lines, standard boilerplate, instructor template code and lines common to most submissions are excluded. Java, C, C++ and JavaScript structure comes from a lightweight parser, and the semantic signal is a rule-based model, not a neural network. No automated method can detect every form of plagiarism.